

Annex C: Free Quantum Resource Guide

Challenge 2: The Quantum Routing Brain

What is free, what costs money, realistic limits, and the recommended hackathon stack. All information current as of April 2026.

FREE - TIER QUANTUM BACKENDS

IBM Quantum Open Plan

What you get: 10 minutes of QPU runtime per 28-day rolling window on `ibm_kingston` (Heron r2, 156 physical qubits). A March 2026 promotional offer gives opted-in users 180 minutes over 12 months.

Hardware: IBM Heron r2 processor. 156 superconducting transmon qubits. Native two-qubit gate: CZ. Median two-qubit gate error $\sim 0.3\text{--}0.5\%$. $T_1/T_2 \sim 100\text{--}300 \mu\text{s}$.

SDK: Qiskit 1.x. Primitives API: `SamplerV2` and `EstimatorV2`. Execution modes: `job` and `batch` (Open Plan users cannot use `session` mode, which means iterative QAOA loops must queue each iteration separately).

Platform: <https://quantum.cloud.ibm.com> (new platform, replaced legacy domain in 2025).

Docs: <https://docs.quantum.ibm.com>

Caveats: No session mode means iterative variational algorithms are slow (each circuit submission enters the queue independently). Plan around this: pre-optimize classically, submit final circuit to QPU for validation rather than running the full optimization loop on hardware. 10 minutes of QPU time is enough for roughly 50–200 small circuit executions depending on depth and shot count.

Best for: Final-stage demo validation. Run your QAOA MaxCut on real hardware for 1–2 problem instances to show the router can hit a real QPU. Do development and iteration on Qiskit Aer locally.

AWS Braket

What you get: 1 hour/month of simulator time (SV1 statevector, DM1 density matrix, TN1 tensor network) on new AWS accounts for 12 months. Zero free QPU shots.

Simulators:

- SV1: statevector, up to 34 qubits, \$0.075/task + \$0.00075/shot (free tier covers this).
- DM1: density matrix with noise, up to 17 qubits.
- TN1: tensor network, up to 50 qubits for shallow circuits.

Hardware (paid only):

- Rigetti Ankaa-3 (superconducting, 84 qubits): \$0.30/task + \$0.0009/shot.
- IQM Garnet (superconducting, 20 qubits): \$0.30/task + \$0.00145/shot.
- IonQ Forte (trapped ion, 36 qubits): \$0.30/task + \$0.01/shot. A 10-qubit MaxCut demo runs roughly \$5–10 on Rigetti/IQM, \$200+ on IonQ.

SDK: Amazon Braket SDK (Python). Hybrid Jobs: `@hybrid_job(device=...)` decorator spins up an EC2 container. Container startup is ≥ 1 minute, too slow for tiny demo workloads. Use `LocalQuantumJob` for development.

Docs: <https://docs.aws.amazon.com/braket>

Best for: Simulator development during the build. SV1 free tier is generous enough for a full week of iteration. If you want to show multi-backend routing (IBM + AWS), use Braket SV1 as the second “quantum backend” alongside IBM Open Plan hardware.

Azure Quantum

What you get: Historically \$500 in free credits per provider (IonQ, Quantinuum, Rigetti, Pasqal). Up to \$10,000 via Research Credits for academic users (application-based).

Hardware: Access to IonQ Aria/Forte, Quantinuum H-series (highest fidelity commercially available), Rigetti, and Pasqal neutral-atom QPUs.

Docs: <https://learn.microsoft.com/en-us/azure/quantum/>

Caveats: Verify credit availability at signup. Terms have changed multiple times since 2023. Quantinuum H-series credits are the most valuable (highest fidelity) but deplete fastest.

Best for: If a team member has an existing Azure academic account with credits. Not recommended as primary platform for a 6-day build due to signup/provisioning overhead.

D-Wave Leap

What you get: 1 minute/month of hybrid solver time. Problems up to millions of binary variables via the Constrained Quadratic Model (CQM) hybrid solver.

Hardware: D-Wave Advantage2 quantum annealer (1,200+ qubits). Not gate-model. Solves QUBO/Ising problems natively. Hybrid solvers combine classical heuristics with quantum annealing.

SDK: Ocean SDK (Python). `dwave-system`, `dimod`, `dwave-hybrid`.

Docs: <https://docs.ocean.dwavesys.com>

Example repos: <https://github.com/dwave-examples/job-shop-scheduling-cqm>

Caveats: D-Wave is an annealer, architecturally different from gate-model QPUs (IBM, IonQ, Rigetti). Your router should treat it as a distinct backend type. 1 minute of hybrid solver time is enough for roughly 10–30 small problem submissions.

Best for: Teams choosing Thread 4 (Logistics/VRP) or any scheduling-heavy thread. The hybrid solver handles 5-task/3-machine scheduling problems trivially. Good demo contrast: show the same QUBO routed to D-Wave annealer vs. Qiskit QAOA on gate-model vs. classical Gurobi.

LOCAL SIMULATORS (NO ACCOUNT REQUIRED)

Qiskit Aer

What it is: IBM's local quantum circuit simulator. Ships with Qiskit. Statevector, density matrix, and stabilizer backends. Supports noise models calibrated from real IBM hardware.

Qubit limits: Statevector simulation requires $2^n \times 16$ bytes of RAM. Practical limits: ~27–30 qubits on a 16 GB laptop, ~32 qubits on 64 GB. Tensor network methods (via `aer_simulator_extended_stabilizer`) handle hundreds of qubits for shallow circuits.

Install: `pip install qiskit-aer`

Best for: Primary development backend. All iteration during the build should happen here. Zero cost, zero queue, instant results.

PennyLane + Lightning

What it is: Xanadu's quantum ML framework. `lightning.qubit` is a high-performance C++ statevector simulator. PennyLane Catalyst (`@qjit` decorator) provides MLIR/LLVM compilation for quantum circuits.

Key advantage: PennyLane ships in the default AWS Braket Hybrid Jobs container, making it the most hackathon-friendly stack for teams that want to develop locally and deploy to Braket cloud with zero configuration changes.

Qubit limits: Same as Qiskit Aer (RAM-bounded statevector). `lightning.gpu` extends to larger circuits with NVIDIA GPU acceleration.

Install: `pip install pennylane pennylane-lightning`

Docs: <https://pennylane.ai/qml/>

Best for: Teams interested in quantum kernel methods / QML for classification tasks. PennyLane's `qml.kernels` module and quantum kernel tutorials are the cleanest path to a QSVM demo.

Google Cirq + qsim

What it is: Google's open-source quantum circuit framework. `qsim` is a high-performance statevector simulator optimized for Google hardware topologies.

Hardware access: None. Google does not provide public access to Sycamore or Willow processors. Cirq is simulator-only for external users.

Install: `pip install cirq qsimcirq`

Docs: <https://quantumai.google/cirq>

Best for: Teams that prefer Cirq's API style. Not recommended as primary unless you have specific experience with it, since Qiskit and PennyLane have larger tutorial ecosystems and cloud backend integrations.

RECOMMENDED HACKATHON STACK

Primary development: PennyLane + `lightning.qubit` on Python 3.12, containerized. Develop locally. All iteration happens on the simulator.

Demo backends: AWS Braket SV1 simulator (free tier, shows cloud offload) + IBM Open Plan Heron r2 (shows real QPU execution). This gives your router two distinct quantum backends to dispatch to, demonstrating multi-backend orchestration.

Optional third backend: D-Wave Leap hybrid solver (shows annealer vs. gate-model routing decision). Especially strong for scheduling/VRP threads.

Wrap as REST: Each backend runs as a containerized FastAPI service so the router sees a uniform `POST /solve` interface regardless of whether the backend is local Aer, cloud Braket, real IBM QPU, or D-Wave annealer.

REFERENCE TUTORIALS AND DEMO CODE

Start with one of these to get a working quantum kernel inside your orchestration layer. Do not build from scratch.

Tutorial	URL	Notes
IBM Qiskit QAOA Patterns (MaxCut)	https://learning.quantum.ibm.com/tutorial/quantum-approximate-optimization-algorithm	Designed to fit Open Plan time limits. Uses <code>SamplerV2/EstimatorV2</code> . Scales from 5 to 100 nodes. Start here.
PennyLane QAOA MaxCut	https://pennylane.ai/qml/demos/tutorial_qaoa_maxcut	Clean <code>qml.qaoa.maxcut(graph)</code> API. Last updated Oct 2025.
AWS Braket QAOA notebook	https://github.com/amazon-braket/amazon-braket-examples	Canonical Braket example. Runs on SV1 free tier.
D-Wave Job Shop Scheduling	https://github.com/dwave-examples/job-shop-scheduling-cqm	Modern CQM repo. 5 tasks / 3 machines fits the 1-min free tier easily.
PennyLane Quantum Kernels	https://pennylane.ai/qml/demos/tutorial_kernels_module	Best path to a QSVM classifier demo for teams on Thread 1 (C-UAS emitter classification).
Tabi et al. Graph Coloring	arXiv:2009.07314	Space-efficient QAOA encoding for frequency assignment. 5 nodes / 3 colors = 15 qubits. Directly maps to Thread 2 spectrum deconfliction analog.

Demo framing. The tutorial code is your quantum kernel. Your value-add is the orchestration layer *around* it: the router that decides when to invoke it, the policy engine that gates by classification and latency, the audit log that records the decision, and the fallback that runs classical when quantum is unavailable. Judges will not be impressed by a copy-pasted QAOA notebook. Judges will be impressed by a router that intelligently dispatches to that notebook under the right conditions and refuses to under the wrong ones.

COST REFERENCE (IF YOU SPEND MONEY)

All free-tier resources are sufficient for the Dry Dock build. If your team chooses to spend money for a more impressive demo, here are approximate costs for small problem instances:

Backend	Cost Model	10 - Qubit MaxCut Demo
IBM Open Plan	Free (10 min/28 days)	Free. ~2–5 seconds of QPU time per execution.
IBM Pay - As - You - Go	\$96/min QPU time	~\$3–8 per execution.
AWS Braket SV1 simulator	Free tier (1 hr/mo)	Free.
AWS Braket Rigetti Ankaa-3	\$0.30/task + \$0.0009/shot	~\$5 for 1,000 shots.
AWS Braket IonQ Forte	\$0.30/task + \$0.01/shot	~\$200+ for 1,000 shots. Avoid unless funded.
D-Wave Leap hybrid	Free (1 min/mo)	Free. Hybrid solver handles small scheduling in seconds.

Budget recommendation: Spend \$0. The free tiers are sufficient. If you want a single real-hardware data point for the demo, IBM Open Plan is free and D-Wave Leap is free. AWS Braket Rigetti at \$5 is the cheapest paid option for a gate-model QPU run. IonQ is not worth the cost for a hackathon.